

視窗程式設計專題報告

復古未來風格即時天氣播報系統

一、專題介紹與製作動機

本專題名稱為「復古未來風格即時天氣播報系統」，英文名稱為 Arcadia Weather Terminal。這是一個使用 Python 製作的桌面應用程式，主要功能是讓使用者可以搜尋世界各地的城市，並取得該城市的即時天氣、每小時預報、未來天氣趨勢、周邊觀測資料、地圖與降雨雷達資訊。除了基本的天氣查詢功能之外，本系統也加入復古未來風格的介面設計、CRT 螢幕效果、開機動畫、背景音樂、音效、廣告頁面，以及本地 AI 天氣總結功能，使整體作品不只是單純的天氣工具，而是一個具有情境感與世界觀的互動式氣象終端機。

本專題的製作動機來自於一般天氣 App 或天氣網站的介面大多偏向實用性，雖然能快速提供溫度、濕度、風速、降雨機率等資訊，但視覺呈現通常較為制式。例如常見的天氣介面多半使用白色背景、圓角卡片、簡單圖示與列表資訊，雖然清楚，但比較缺少記憶點與沉浸感。因此，我希望嘗試將一個日常生活中常見的功能，也就是天氣查詢，轉換成更具有設計感和互動感的系統。

本系統的核心想法是「把真實天氣資料包裝成復古未來世界中的資訊終端」。使用者在操作時，不只是單純輸入城市並看到天氣數字，而是像在操作一台來自科幻殖民地的氣象設備。這樣的設計讓天氣資料不只是資料，而是成為畫面敘事的一部分。

在美術風格上，本專題主要參考遊戲 The Outer Worlds 的復古未來與企業殖民地風格。該遊戲的美術特色不是單純現代高科技，而是結合老式廣告、企業宣傳、殖民地終端機、復古商標與略帶諷刺感的公司標語。因此，本系統在介面中加入暗色背景、黃銅色邊框、橘色與綠色字體、復古字型、企業廣告素材、老式電視外框以及 CRT 螢幕掃描效果，讓整體畫面更接近復古科幻終端機的感覺。

本專題的目標可以分為三個層面。第一個層面是功能性，也就是系統必須能取得真實天氣資料，而不是使用手動輸入或固定假資料。第二個層面是資訊整理，因為 API 回傳的資料通常是大量數值、代碼與陣列，使用者不容易直接理解，因此程式需要將這些資料轉換成文字、表格、圖表與地圖。第三個層面是視覺風格，讓整個系統不只是工具，也能成為一個具有主題性的互動作品。

二、系統功能與操作流程

本系統啟動後，使用者會先看到首頁與城市搜尋介面。首頁是整個系統的入口，使用者可以在搜尋欄輸入城市名稱，例如 Taipei、Seattle、Tokyo、London 等。輸入城市後，程式會先透過城市搜尋 API 找到該城市對應的經緯度。這一步是後續所有資料取得的基礎，因為大部分天氣 API 並不是直接使用城市名稱查詢，而是透過 latitude 和 longitude，也就是緯度與經度，來定位實際地點。

取得座標後，系統會根據該城市的位置載入相關資料，包含即時天氣資料、每小時預報、未來預報、地圖底圖、降雨雷達圖層，以及 AI 天氣總結需要的資料。搜尋成功後，系統會從第一個天氣頁面開始播放，避免重新搜尋後還停留在上一個城市的頁面。這樣的設計可以降低新舊資料混在一起的問題，也讓使用者更清楚目前畫面顯示的是哪一個城市。

系統主要採用輪播式頁面設計，每隔一段時間自動切換不同頁面。使用者也可以透過底部控制列操作，例如上一頁、下一頁、暫停、全螢幕、重新整理與重新搜尋城市。這些控制功能是為了讓系統更適合 Demo 展示。例如在報告時，可以按下暫停停留在某一頁，方便講解該頁面的資料來源、視覺設計與功能原理。

主要頁面包含 Current Weather、Weather Summary、Hourly Forecast、Forecast、Radar Map、Colony Comparison Board、Local Observations 與 Advertisement Page。Current Weather 頁面顯示目前城市的主要天氣資訊，例如溫度、天氣描述、濕度、風速與能見度。這是使用者搜尋後最先看到的主要資料頁，目的是快速回答「現在這個城市天氣如何」。

Weather Summary 頁面則是將目前城市的天氣資料整理後，交給本機 AI 模型 Ollama 產生英文天氣總結。這段總結會以天氣狀況為主，同時加入復古企業廣播的語氣，使它更符合整體風格。

Hourly Forecast 頁面顯示接下來幾個小時的天氣變化，主要包含溫度走勢與降雨機率。

Forecast 頁面則偏向未來幾天的天氣趨勢。Radar Map 頁面會根據搜尋城市的經緯度顯示地圖與降雨雷達，使使用者能以更直觀的方式觀察該地區附近是否有雨區。

另外，Colony Comparison Board 頁面用來比較多個城市的即時天氣，Local Observations 則顯示搜尋城市附近的周邊地區天氣。Advertisement Page 是復古廣告頁，目的不是提供天氣資料，而是補強整體世界觀，讓系統更像存在於科幻世界中的資訊終端。

三、設計理念與視覺風格

本專題的設計理念是將「真實資料」與「虛構世界觀」結合。天氣資料本身是客觀且生活化的資訊，例如溫度、風速、濕度與降雨機率，但如果只是用一般表格或卡片呈現，作品會比較像普通工具。因此，本系統嘗試將這些資料轉換成一個有角色感、有風格、有情境的終端機介面。

在視覺風格上，本系統選擇復古未來方向，而不是現代科技風。一般科技介面常使用藍色光線、透明玻璃效果與極簡圖示，但本專題希望呈現的是比較老式、粗糙、有企業殖民地感的科幻設備。因此，整體介面使用深色底色搭配黃銅色、橘色、青綠色與暗綠色，模擬舊式螢幕與工業設備的視覺印象。

字體也是視覺風格的重要部分。系統使用自訂字體，使畫面看起來不像一般系統預設介面，而更接近遊戲中的終端機文字。畫面外框則設計成老式電視或 CRT 螢幕的感覺，搭配角落裝飾、掃描線、低亮度背景網格和跑馬燈，使畫面更有年代感。

動畫設計方面，系統加入開機動畫、頁面轉場、底部跑馬燈、雷達掃描線和 CRT 氣氛層。這些效果主要使用 Tkinter Canvas 繪製，並透過 `after()` 定時更新畫面。例如雷達頁的掃描線會持續旋轉，模擬氣象雷達正在運作；底部跑馬燈會持續移動，模擬企業公告或警示訊息；頁面切換時會有短暫轉場，讓整體不會像靜態簡報，而更像一個正在運行的系統。

不過，在設計過程中也發現視覺效果不能過度。早期版本的掃描線與黑色外框比較強，雖然復古感更明顯，但會蓋住文字或圖表，造成閱讀困難。後來有降低掃描線強度，並移除會穿過內容的大面積黑框，只保留邊緣氣氛與角落效果。這讓畫面保有復古未來風格，同時維持資訊可讀性。

廣告頁也是設計理念的一部分。因為 The Outer Worlds 的世界觀中常出現企業廣告與諷刺式標語，所以本系統加入復古廣告圖片與企業口吻宣傳標語。這些頁面讓系統不只是顯示資料，而像是一台真的存在於遊戲世界中的公共資訊終端。

四、使用的函式庫與工具

本專題主要使用 Python 製作，並搭配多個函式庫與工具完成不同功能。這些函式庫分別負責介面、資料請求、圖片處理、音效播放、背景執行與 AI 文字生成。

首先是 Tkinter。Tkinter 是 Python 常見的 GUI 工具，本系統的主要視窗、搜尋欄、按鈕、頁面切換、全螢幕模式與 Canvas 繪圖都使用 Tkinter 完成。其中 Canvas 是本系統最重要的繪圖工具，因為畫面中的文字、表格、折線圖、長條圖、雷達掃描線、邊框、轉場動畫與 CRT 效果，大多都是透過 Canvas 即時繪製出來。

第二個是 requests。requests 主要負責和外部 API 溝通。系統需要透過網路向 Open-Meteo 取得城市搜尋與天氣資料，也需要向 OpenStreetMap 取得地圖瓦片，向 RainViewer 取得降雨雷達圖層。這些 HTTP 請求都可以透過 requests 完成。API 回傳資料後，程式會把 JSON 格式資料轉換成 Python 字典或列表，再進一步整理成畫面要顯示的內容。

第三個是 Pillow，也就是 PIL。Pillow 主要負責圖片處理。系統中的廣告圖片、Logo、地圖底圖與雷達圖層，都需要經過縮放、裁切、透明度處理或風格化處理，才能符合整體畫面需求。例如地圖原本是一般 OpenStreetMap 的彩色地圖，程式會透過 Pillow 將它轉成較暗、較復古的終端機風格，避免和整體 UI 風格不一致。

第四個是 pygame。pygame 在本專題中主要負責音樂與音效播放，例如背景音樂、開機音效與操作提示音。雖然 pygame 常被用於遊戲製作，但在本系統中，它主要是作為音訊播放工具，讓整個終端機更有氛圍。

第五個是 threading。因為 API 請求、地圖載入、雷達圖層下載和圖片處理都可能需要時間，如果全部在主執行緒中執行，Tkinter 視窗可能會卡住。因此程式使用 threading 讓部分工作在背景執行，降低介面卡頓的機率。這對於需要即時動畫和跑馬燈的系統非常重要。

最後是 Ollama。Ollama 不是 Python 函式庫，而是一個本地 AI 工具。程式會把目前城市的天氣資料整理成 prompt，送到 Ollama 產生英文天氣總結。使用 Ollama 的優點是不用付費雲端 AI API，也不需要額外申請金鑰；缺點是產生速度會受到電腦效能影響，而且使用前需要先啟動本機服務。

本專題主要使用的工具如下：

```
import tkinter as tk
import requests
import threading
import pygame
from PIL import Image, ImageTk
```

Tkinter 負責 GUI 介面，requests 負責 API 請求，threading 讓資料載入可以在背景執行，pygame 負責音效播放，Pillow 則負責圖片處理。

五、API 資料取得與資料呈現方式

本系統取得資料的流程可以簡化為「城市名稱 → 經緯度 → 天氣資料 → 資料整理 → 畫面呈現」。使用者輸入城市名稱後，程式會先透過 Open-Meteo Geocoding API 查詢對應地點的經緯度。取得經緯度後，再使用 Open-Meteo Forecast API 查詢該位置的即時天氣、每小時預報與未來預報。

API 回傳的資料通常是 JSON 格式，其中包含大量欄位與數值。例如 temperature_2m 代表地表上方 2 公尺高度的氣溫，這是氣象資料常用的標準高度，因為它比較接近一般人實際感受到的空氣溫度。precipitation_probability 則代表降雨機率，通常以百分比呈現，例如 20%、50% 或 80%。weather_code 則是天氣狀態代碼，例如晴天、陰天或下雨都可能用不同數字表示。

程式不會直接把這些原始資料顯示給使用者，而是會先整理。例如 weather_code 對一般使用者來說不容易理解，因此程式會將它轉換成 Clear Sky、Overcast、Rain、Drizzle 等英文描

述。溫度會加上攝氏單位，濕度會轉換成百分比，風速會顯示為 km/h，使資訊更容易閱讀。

以下是簡化版的 API 請求範例：

```
import requests

url = "https://api.open-meteo.com/v1/forecast"

params = {
    "latitude": latitude,
    "longitude": longitude,
    "current": [
        "temperature_2m",
        "relative_humidity_2m",
        "wind_speed_10m",
        "weather_code"
    ],
    "hourly": [
        "temperature_2m",
        "precipitation_probability"
    ]
}

response = requests.get(url, params=params, timeout=10)
weather_data = response.json()
```

這段程式碼示範程式如何使用城市經緯度向 Open-Meteo Forecast API 請求天氣資料。其中 temperature_2m 代表地表上方 2 公尺高度的氣溫，precipitation_probability 代表降雨機率。

以下是將天氣代碼轉換成文字的簡化概念：

```
def weather_code_to_text(code):
    mapping = {
        0: "Clear Sky",
        1: "Mainly Clear",
        2: "Partly Cloudy",
        3: "Overcast",
        61: "Rain",
        80: "Rain Showers"
    }
    return mapping.get(code, "Unknown")
```

透過這些處理，API 回傳的原始資料才能變成使用者看得懂的天氣資訊。

在畫面呈現方面，不同頁面會使用不同形式。Current Weather 頁面以文字與資訊區塊顯示目前天氣。Hourly Forecast 頁面會將 temperature_2m 轉換成折線圖，讓使用者看到接下來幾小時的溫度變化；同時將 precipitation_probability 轉換成長條圖，表示不同時段的降雨機率。Forecast 頁面則整理未來幾天的最高溫、最低溫與降雨風險。

Radar Map 頁面使用 OpenStreetMap 作為地圖底圖，並疊加 RainViewer 的降雨雷達圖層。這部分會根據搜尋城市的座標更新，因此搜尋不同城市時，地圖位置會跟著改變。需要特別說明的是，地圖和 RainViewer 雷達圖層是真實資料，而旋轉掃描線和彩色圓圈是程式額外做的視覺化效果。這些圓圈會參考降雨機率和天氣資料，但不完全等同於官方氣象雷達回波，主要目的是強化復古雷達介面的視覺效果。

六、困難與解決方法

在開發過程中，第一個困難是資料更新不同步。因為本系統有許多頁面，每一頁使用的資料不同，例如即時天氣、AI 總結、每小時預報、地圖雷達、周邊觀測和多城市比較。因此當使用者重新搜尋城市時，不只城市名稱要更新，所有頁面的資料都要同步更新。早期版本曾經出現某些頁面還顯示上一個城市資料的情況，例如地圖沒有更新、跑馬燈卡住或 City Lock 還停在舊城市。後來的解決方法是讓每次搜尋成功後，都重置目前資料狀態，並且從第一頁重新開始播放，減少新舊資料混在一起的問題。

第二個困難是搜尋和載入速度較慢。搜尋一次城市時，系統其實需要處理很多資料，包括城市座標、即時天氣、每小時預報、未來預報、周邊地區、地圖、雷達和 AI 總結。這些資料來源有些來自網路 API，有些需要本機 AI 生成，所以等待時間有時會比較長。為了解決這個問題，系統加入進度條，讓使用者知道程式正在載入資料，而不是當掉。未來若繼續改進，可以改成分階段載入，先顯示主要天氣，再背景載入地圖、雷達與 AI 總結，也可以加入快取機制，加快重複搜尋同一城市時的速度。

第三個困難是 AI 語音播報。最初曾經希望加入 AI 語音，讓系統可以用英文播報天氣，並模仿 The Outer Worlds 那種企業廣播感。但實際測試後發現，免費語音工具的聲音比較機械，不夠自然，也較難表現出需要的語氣。較自然的語音 API 通常需要付費或 API Key，對學生專題來說成本較高。因此最後選擇移除 AI 語音播報，保留文字版 AI Weather Summary，讓展示效果更穩定。

第四個困難是視覺效果與可讀性之間的平衡。復古 CRT 效果可以讓畫面更有風格，但如果掃描線、黑框、雜訊或轉場效果太強，就會蓋住天氣數字和圖表。早期版本曾經出現背景橫線影響閱讀、淺黃色文字搭配白色背景不清楚、圖表線條蓋住溫度標籤等問題。後來透過降低掃描線強度、調整顏色對比、改變圖層繪製順序，以及移除干擾內容的大面積黑框，使畫面更清楚。

第五個困難是全螢幕模式與雷達圖穩定性。由於 Tkinter Canvas 在全螢幕時需要重新縮放畫面，如果縮放邏輯處理不好，外框或雷達圖可能會出現跳動、偏移或比例不一致。後來透過調整 Canvas 的縮放計算方式，並讓雷達元素依照全螢幕實際比例繪製，減少畫面跳動問題。

七、目前限制與未來改進

目前系統已經可以完成主要功能，但仍有一些限制。第一，天氣資料和 Apple Weather、Google Weather 等平台可能有差異。這並不代表 Open-Meteo 資料是假的，而是因為不同平台使用的資料來源、預報模型、更新時間與定位方式不同。Open-Meteo 的優點是免費、開放、免 API Key，適合學生專題與 GitHub 分享；但某些商業天氣服務可能會整合更多在地測站或付費資料，因此在部分地區可能會更接近使用者實際感受。

第二，AI Summary 功能依賴 Ollama。如果 Ollama 沒有啟動，AI 總結就無法產生。雖然程式有錯誤提示，但現場 Demo 前仍需要先確認 Ollama 是否正常執行。未來可以加入更完整的 fallback summary，也就是當 AI 無法使用時，系統仍能根據模板產生普通版天氣總結。

第三，雷達頁中有真實資料，也有視覺化輔助。地圖和 RainViewer 雷達圖層是真實資料，會根據城市座標更新；但旋轉掃描線和彩色圓圈是程式額外產生的效果。這些效果主要是為了強化復古雷達介面，不完全代表官方雷達回波。未來如果要更精準，可以讓畫面更清楚標示哪些是真實雷達資料，哪些是視覺化效果。

第四，搜尋速度仍有優化空間。因為系統整合很多資料來源，每次搜尋需要等待多個 API 與圖片載入。未來可以透過快取、非同步請求、分階段載入和更好的 loading 狀態設計，讓使用者更快看到主要資訊。

未來如果有更多時間，可以加入更自然的 AI 語音播報，使用較高品質的 TTS API；也可以讓使用者自訂多城市比較清單，或加入天氣警報功能，例如暴雨、高溫、低溫、颱風提醒等。這些功能可以讓系統不只是展示型作品，也更接近實際可使用的氣象資訊平台。

八、個人心得與結論—S1254032黃品皓

透過這次專題，我學到最多的是如何把一個普通功能轉換成一個完整的互動系統。一開始以為天氣系統只是查 API 並顯示資料，但實際製作後才發現，真正困難的地方在於資料整理、狀態同步、畫面設計和使用體驗。API 回傳的資料通常只是數字、代碼和陣列，如果沒有經過整理，使用者很難直接理解。因此，如何把資料轉換成清楚的文字、圖表和地圖，是這次專題很重要的學習。

我也學到視覺風格和功能之間需要平衡。復古未來風格可以讓作品更有特色，但如果過度追求風格，可能會讓畫面太花、文字太難讀，反而影響使用。因此在開發過程中，不只是加入效果，也需要不斷刪除或調整效果。例如掃描線、外框、轉場和雷達動畫，都需要在美術感和可讀性之間取得平衡。

另外，這次專題也讓我更了解 Python 桌面應用程式的結構。Tkinter 負責介面，requests 負責 API，Pillow 負責圖片處理，pygame 負責音效，threading 負責背景工作，Ollama 負責 AI 文字生成。這些工具各自負責不同任務，最後再整合成一個完整系統。這讓我理解到一個專案不只是單一功能，而是許多模組互相配合的結果。

在團隊分享方面，這個專題也整理成 GitHub 專案，並將素材改成相對路徑，讓組員可以下載後直接執行。這讓我學到專案不只要在自己的電腦能跑，也要考慮其他人如何安裝、如何啟動，以及是否會因為缺少素材或路徑不同而出錯。

總結來說，本專題完成了一個復古未來風格的即時天氣播報系統。它整合了城市搜尋、即時天氣、每小時預報、未來預報、地圖雷達、AI 天氣總結、音效和廣告頁面，並透過 The Outer Worlds 風格的終端機介面呈現。這次作品讓我學到 API 串接、GUI 設計、資料視覺化、圖片處理與互動設計，也讓我理解到完成一個系統需要兼顧功能、穩定性、視覺風格與使用者體驗。未來如果有更多時間，我希望能繼續優化載入速度、AI 語音播報和雷達精準度，讓它更接近真正完整的科幻氣象終端機。